



Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Московский государственный технический университет имени
Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ «ИУ Информатика и системы управления»

КАФЕДРА «ИУ7 Программное обеспечение ЭВМ и информационные технологии»

Лабораторная работа № 7 по дисциплине «Анализ алгоритмов»

Тема Конечные автоматы

Студент Еланская Е.Д.

Группа ИУ7-53Б

Преподаватели Волкова Л.Л., Строганов Ю.В.

Москва, 2026

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	3
1 Аналитическая часть	4
1.1 БЯМ	4
1.2 Задание по варианту	4
2 Конструкторская часть	5
2.1 Qwen3-coder	5
2.2 Devstral-2	5
2.3 Cogito-2.1	5
2.4 Gemma3	5
3 Технологическая часть	6
3.1 Исходный запрос	6
3.2 Результат запросов	6
4 Исследовательская часть	17
4.1 Тестирование	17
4.2 Трудоёмкость	17
ЗАКЛЮЧЕНИЕ	18
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	19

ВВЕДЕНИЕ

Цель работы – разработать и реализовать программное обеспечение на языке python для извлечения данных из текстовых файлов, полученных из pdf-файлов с применением библиотеки PyPDF2, с использованием регулярных выражений.

Задачи, необходимые для достижения поставленной цели:

- 1) разработать регулярные выражения для решения задачи по варианту;
- 2) реализовать функцию для поиска подстроки по варианту в pdf-файле с использованием разработанных регулярных выражений (привести листинг как самой функции, так и списка необходимых для её функционирования библиотек, а также пример вызова функции), возвращающую кортеж вида:
 - значение, определяющее наличие строки согласно заданному регулярному выражению ("истина/ложь");
 - список кортежей (если "ложь" то список пустой) с двумя составляющими каждого кортежа — строкой, найденной с помощью регулярного выражения, и координатами для определения найденной строки в документе (минимально необходимо указать номер страницы и номер строки документа, в которой найдено вхождение искомой строки. непосредственно таблицы и изображения строками не считать, но их названия считаются строками);
- 3) реализовать программное обеспечение, принимающее на вход путь к pdf-файлу, использующее функцию из предыдущего пункта;
- 4) проверить реализацию на приложенных к лабораторной работе файлах, привести таблицу с колонками "название использованного pdf-файла" "признак успешного нахождения подстроки" "координаты первого нахождения подстроки". необходимо приложить файлы, в которых были найдены ошибки, в репозиторий лабораторной работы.

1 Аналитическая часть

Конечный автомат – модель с конечным набором состояний, переходами между ними по входным символам, начальным состоянием и принимающими состояниями. Детерминированный конечный автомат (ДКА) для каждого состояния и символа имеет ровно один переход, что позволяет эффективно обрабатывать последовательности.

Регулярное выражение – формальная нотация для описания регулярных языков, эквивалентная конечным автоматам. Оно строится из базовых операций: конкатенация, альтернатива ($|$), звезда Клини ($*$), используя символы алфавита [2].

1.1 БЯМ

Для получения реализаций по заданию использовались следующие языковые модели [1]:

- qwen3-coder – модель от Alibaba, оптимизированная для кодирования и агентных задач с длинным контекстом, обеспечивает высокую производительность в генерации и анализе кода;
- devstral-2 – Agentic LLM для инженерии ПО, превосходит в работе с инструментами, исследовании кодовых баз и редактировании нескольких файлов;
- cogito-2.1 – модель для когнитивных задач и RAG-систем) с фокусом на обработку знаний;
- gemma3 – компактная модель от Google, эффективна для общих задач ИИ, включая кодирование и рассуждения.

1.2 Задание по варианту

Проверка на наличие ошибки использования термина «оглавление» вместо «содержание».

Вывод

В данном разделе мной были выбраны БЯМ для решения задания по варианту.

2 Конструкторская часть

2.1 Qwen3-coder

К изначальной формулировке задания пришлось добавить «не использовать regex». Без этого уточнения запрос про конечный автомат игнорировался.

2.2 Devstral-2

К запросу пришлось добавить формат вывода результата при нахождении «оглавление» вместо «содержание».

2.3 Cogito-2.1

Пришлось повторить исходный запрос дважды, так как первая программа оказалась нерабочей.

2.4 Gemma3

Исходный запрос оказался достаточным для получения необходимого результата.

Вывод

В данном разделе мной был описан процесс взаимодействия с каждой выбранной БЯМ в рамках решения задачи.

3 Технологическая часть

3.1 Исходный запрос

Напиши код на python3, который определяет, используется ли в указанном пользователем pdf файле оглавление вместо содержание (могут быть заглавными). Для определения использовать конечный автомат, не regex. Для чтения использовать PyPDF2. Если используется оглавление вместо содержание, то вывод в формате (строка, страница, строка).

3.2 Результат запросов

```
import PyPDF2

def find_oglavlenie_instead_of_soderzhanie(pdf_path):
    """
    Определяет использование слова "оглавление" вместо "содержание" в PDF файле
    Использует конечный автомат для поиска
    """

    # Конечный автомат для поиска слова "оглавление"
    class OglavlenieAutomaton:
        def __init__(self):
            self.state = 0
            self.target_word = "оглавление"
            self.target_word_upper = "ОГЛАВЛЕНИЕ"
            self.found_positions = []
            self.current_char_pos = 0
            self.current_page = 1
            self.current_line = 1
            self.char_in_line = 0
            self.line_start_pos = 0

        def reset(self):
            self.state = 0

        def process_char(self, char, page_num, char_pos_in_text, absolute_pos):
            # Отслеживаем строки
            if char == '\n':
                self.current_line += 1
                self.char_in_line = 0
                self.line_start_pos = absolute_pos + 1
            else:
                self.char_in_line += 1

            # Проверяем обычное написание
            if self.state < len(self.target_word):
                if char.lower() == self.target_word[self.state]:
                    self.state += 1
                    if self.state == len(self.target_word):
                        # Найдено слово "оглавление"
                        line_num = self.current_line
                        char_pos_in_line = self.char_in_line - len(self.target_word) + 1
                        if char_pos_in_line < 1:
                            char_pos_in_line = 1
                        self.found_positions.append({
```

```

        'word': 'оглавление',
        'page': page_num,
        'line': line_num,
        'char_pos': char_pos_in_line
    })
    self.reset()
else:
    self.reset()
    # Проверяем с текущего символа
    if char.lower() == self.target_word[0]:
        self.state = 1

# Проверяем заглавные буквы
if self.state == 0: # Только если не в процессе поиска обычного слова
    temp_state = 0
    if temp_state < len(self.target_word_upper):
        if char == self.target_word_upper[temp_state]:
            temp_state += 1
        # Продолжаем проверку для остальных символов
        # Это упрощенная проверка, в реальном автомате нужно более сложное состояние

# Конечный автомат для поиска слова "содержание"
class SoderzhanieAutomaton:
    def __init__(self):
        self.state = 0
        self.target_word = "содержание"
        self.target_word_upper = "СОДЕРЖАНИЕ"

    def reset(self):
        self.state = 0

    def process_char(self, char):
        if self.state < len(self.target_word):
            if char.lower() == self.target_word[self.state]:
                self.state += 1
                if self.state == len(self.target_word):
                    self.reset()
                    return True
            else:
                self.reset()
                if char.lower() == self.target_word[0]:
                    self.state = 1
        return False

try:
    with open(pdf_path, 'rb') as file:
        pdf_reader = PyPDF2.PdfReader(file)
        results = []

    for page_num in range(len(pdf_reader.pages)):
        page = pdf_reader.pages[page_num]
        text = page.extract_text()

        # Очищаем текст от лишних пробелов
        lines = text.split('\n')

        # Используем конечный автомат для поиска
        oglavlenie_state = 0
        soderzhanie_state = 0

```

```

oglavlenie_target = "оглавление"
soderzhanie_target = "содержание"
oglavlenie_target_upper = "ОГЛАВЛЕНИЕ"
soderzhanie_target_upper = "СОДЕРЖАНИЕ"

char_position = 0
line_number = 1
char_in_line = 0
line_start_positions = [0]

# Подсчитываем позиции строк
for i, char in enumerate(text):
    if char == '\n':
        line_number += 1
        line_start_positions.append(i + 1)

line_number = 1
for i, char in enumerate(text):
    if char == '\n':
        line_number += 1
        char_in_line = 0
    else:
        char_in_line += 1

# Проверяем "оглавление"
found_oglavlenie = False
if oglavlenie_state < len(oglavlenie_target):
    if char.lower() == oglavlenie_target[oglavlenie_state]:
        oglavlenie_state += 1
        if oglavlenie_state == len(oglavlenie_target):
            found_oglavlenie = True
else:
    # Проверяем заглавными
    if char == oglavlenie_target_upper[0] and oglavlenie_state == 0:
        # Начинаем проверку заглавного варианта
        temp_state = 1
        j = 1
        while (i + j < len(text) and
               temp_state < len(oglavlenie_target_upper) and
               text[i + j] == oglavlenie_target_upper[temp_state]):
            temp_state += 1
            j += 1
        if temp_state == len(oglavlenie_target_upper):
            found_oglavlenie = True
            oglavlenie_state = 0
            # Найдено "ОГЛАВЛЕНИЕ"
            line_num = line_number
            char_pos_in_line = char_in_line - len(oglavlenie_target_upper) + 1
            if char_pos_in_line < 1:
                char_pos_in_line = 1
            results.append((line_num, page_num + 1, char_pos_in_line))
            # Пропускаем обработанные символы
            continue

oglavlenie_state = 0
if char.lower() == oglavlenie_target[0]:
    oglavlenie_state = 1

```

```

if found_oglavlenie and oglavlenie_state == len(oglavlenie_target):
    # Проверяем, есть ли "содержание" поблизости (в пределах нескольких символов)
    # Это простая проверка - в реальном случае нужно более точное определение

    # Находим позицию строки
    line_num = line_number
    char_pos_in_line = char_in_line - len(oglavlenie_target) + 1
    if char_pos_in_line < 1:
        char_pos_in_line = 1

    # Проверяем, не было ли "содержание" недавно
    soderzhание_found_recently = False
    check_start = max(0, i - 50) # Проверяем 50 символов назад
    temp_state = 0
    for j in range(check_start, i):
        if temp_state < len(soderzhание_target):
            if text[j].lower() == soderzhание_target[temp_state]:
                temp_state += 1
                if temp_state == len(soderzhание_target):
                    soderzhание_found_recently = True
                    break
            else:
                temp_state = 0
                if text[j].lower() == soderzhание_target[0]:
                    temp_state = 1

    # Если "содержание" не найдено недавно, значит это замена
    if not soderzhание_found_recently:
        results.append((line_num, page_num + 1, char_pos_in_line))

    oglavlenie_state = 0

# Проверяем заглавное "ОГЛАВЛЕНИЕ" отдельно
if (oglavlenie_state == 0 and
    i + len(oglavlenie_target_upper) <= len(text)):
    is_upper_oglavlenie = True
    for k in range(len(oglavlenie_target_upper)):
        if text[i + k] != oglavlenie_target_upper[k]:
            is_upper_oglavlenie = False
            break

    if is_upper_oglavlenie:
        # Проверяем контекст на наличие "содержание"
        soderzhание_found_recently = False
        check_start = max(0, i - 50)
        temp_state = 0
        for j in range(check_start, i):
            if temp_state < len(soderzhание_target):
                if text[j].lower() == soderzhание_target[temp_state]:
                    temp_state += 1
                    if temp_state == len(soderzhание_target):
                        soderzhание_found_recently = True
                        break
            else:
                temp_state = 0
                if text[j].lower() == soderzhание_target[0]:
                    temp_state = 1

        if not soderzhание_found_recently:

```

```

        line_num = line_number
        char_pos_in_line = char_in_line + 1
        results.append((line_num, page_num + 1, char_pos_in_line))

        # Пропускаем обработку этих символов
        oglavlenie_state = 0
        continue

    return results

except Exception as e:
    print(f"Ошибка при обработке PDF файла: {e}")
    return []

def main():
    pdf_path = input("Введите путь к PDF файлу: ").strip()

    # Убираем кавычки, если они есть
    if pdf_path.startswith('\'') and pdf_path.endswith('\''):
        pdf_path = pdf_path[1:-1]
    if pdf_path.startswith('"') and pdf_path.endswith('"'):
        pdf_path = pdf_path[1:-1]

    results = find_oglavlenie_instead_of_soderzhanie(pdf_path)

    if results:
        print("Найдены случаи использования 'оглавление' вместо 'содержание':")
        for line, page, char_pos in results:
            print(f"({line}, {page}, {char_pos})")
    else:
        print("Случаи использования 'оглавление' вместо 'содержание' не найдены")

if __name__ == "__main__":
    main()

```

Листинг 1 – qwen3-coder.

```

import PyPDF2
from typing import List, Tuple, Optional

class ContentChecker:
    def __init__(self):
        # Конечный автомат для поиска "оглавление" или "содержание"
        self.states = {
            'start': {'о': 'о', 'с': 'с'},
            'о': {'г': 'ог'},
            'ог': {'л': 'огл'},
            'огл': {'а': 'огла'},
            'огла': {'в': 'оглав'},
            'оглав': {'л': 'оглавл'},
            'оглавл': {'е': 'оглавле'},
            'оглавле': {'н': 'оглавлен'},
            'оглавлен': {'и': 'оглавлени'},
            'оглавлени': {'е': 'found_оглавление'},
            'с': {'о': 'со'},
            'со': {'д': 'сод'},
            'сод': {'е': 'соде'},
            'соде': {'р': 'содер'},

```

```

        'содер': {'ж': 'содерж'},
        'содерж': {'а': 'содержа'},
        'содержа': {'н': 'содержан'},
        'содержан': {'и': 'содержани'},
        'содержани': {'е': 'found_содержание'}
    }

    self.current_state = 'start'
    self.found_words = []

def reset(self):
    self.current_state = 'start'

def process_char(self, char: str, page_num: int, line_num: int) -> Optional[Tuple[str, int, int]]:
    char = char.lower()
    if char in self.states.get(self.current_state, {}):
        self.current_state = self.states[self.current_state][char]
        if self.current_state.startswith('found_'):
            word = self.current_state.split('_')[1]
            self.reset()
            return (word, page_num, line_num)
        else:
            self.reset()
    return None

def check_pdf_content(pdf_path: str) -> List[Tuple[str, int, int]]:
    checker = ContentChecker()
    results = []

    with open(pdf_path, 'rb') as file:
        reader = PyPDF2.PdfReader(file)

        for page_num in range(len(reader.pages)):
            page = reader.pages[page_num]
            text = page.extract_text()

            if text:
                lines = text.split('\n')
                for line_num, line in enumerate(lines, 1):
                    for char in line:
                        result = checker.process_char(char, page_num + 1, line_num)
                        if result:
                            results.append(result)

    return results

def main():
    pdf_path = input("Введите путь к PDF-файлу: ")
    results = check_pdf_content(pdf_path)

    if not results:
        print("В файле не найдено ни 'оглавление', ни 'содержание'.")
        return

    # Проверяем, есть ли "оглавление" и нет ли "содержание"
    has_оглавление = any(word == 'оглавление' for word, _, _ in results)
    has_содержание = any(word == 'содержание' for word, _, _ in results)

    if has_оглавление and not has_содержание:
        print("В файле используется 'оглавление' вместо 'содержание'.")

```

```

        for word, page, line in results:
            if word == 'оглавление':
                print(f"({word}, {page}, {line})")
    elif has_содержание and not has_оглавление:
        print("В файле используется 'содержание' (не 'оглавление').")
    else:
        print("В файле найдены оба варианта или ни одного.")

if __name__ == "__main__":
    main()

```

Листинг 2 – devstral-2.

```

import PyPDF2
from enum import Enum, auto

class State(Enum):
    SEARCH = auto()
    FOUND_0 = auto()
    FOUND_0G = auto()
    FOUND_OGL = auto()
    FOUND_OGLA = auto()
    FOUND_OGLAV = auto()
    FOUND_OGLAVL = auto()
    FOUND_OGLAVLE = auto()
    FOUND_OGLAVLEN = auto()
    FOUND_OGLAVLENI = auto()
    FOUND_OGLAVLENIE = auto()
    FOUND_S = auto()
    FOUND_SO = auto()
    FOUND_SOD = auto()
    FOUND_SODE = auto()
    FOUND_SODER = auto()
    FOUND_SODERZH = auto()
    FOUND_SODERZHA = auto()
    FOUND_SODERZHAN = auto()
    FOUND_SODERZHANI = auto()
    FOUND_SODERZHANIE = auto()

def find_toc_mentions(pdf_path):
    results = []

    with open(pdf_path, 'rb') as file:
        reader = PyPDF2.PdfReader(file)

        for page_num in range(len(reader.pages)):
            page = reader.pages[page_num]
            text = page.extract_text()

            if not text:
                continue

            lines = text.split('\n')

            for line_num, line in enumerate(lines, 1):
                state = State.SEARCH
                word_start = -1

```

```

for i, char in enumerate(line.lower()):
    if state == State.SEARCH:
        if char == 'o':
            state = State.FOUND_0
            word_start = i
        elif char == 'c':
            state = State.FOUND_S
            word_start = i
        else:
            word_start = -1

# Проверка для "оглавление"
elif state == State.FOUND_0:
    if char == 'r': state = State.FOUND_OG
    else: state = State.SEARCH
elif state == State.FOUND_OG:
    if char == 'n': state = State.FOUND_OGL
    else: state = State.SEARCH
elif state == State.FOUND_OGL:
    if char == 'a': state = State.FOUND_OGLA
    else: state = State.SEARCH
elif state == State.FOUND_OGLA:
    if char == 'b': state = State.FOUND_OGLAV
    else: state = State.SEARCH
elif state == State.FOUND_OGLAV:
    if char == 'n': state = State.FOUND_OGLAVL
    else: state = State.SEARCH
elif state == State.FOUND_OGLAVL:
    if char == 'e': state = State.FOUND_OGLAVLE
    else: state = State.SEARCH
elif state == State.FOUND_OGLAVLE:
    if char == 'n': state = State.FOUND_OGLAVLEN
    else: state = State.SEARCH
elif state == State.FOUND_OGLAVLEN:
    if char == 'n': state = State.FOUND_OGLAVLENI
    else: state = State.SEARCH
elif state == State.FOUND_OGLAVLENI:
    if char == 'e': state = State.FOUND_OGLAVLENIE
    else: state = State.SEARCH
elif state == State.FOUND_OGLAVLENIE:
    if char == ' ' or not char.isalpha():
        results.append(("ОГЛАВЛЕНИЕ", page_num + 1, line_num, line.strip()))
        state = State.SEARCH
    else:
        state = State.SEARCH

# Проверка для "содержание"
elif state == State.FOUND_S:
    if char == 'o': state = State.FOUND_SO
    else: state = State.SEARCH
elif state == State.FOUND_SO:
    if char == 'd': state = State.FOUND_SOD
    else: state = State.SEARCH
elif state == State.FOUND_SOD:
    if char == 'e': state = State.FOUND_SODE
    else: state = State.SEARCH
elif state == State.FOUND_SODE:
    if char == 'p': state = State.FOUND_SODER
    else: state = State.SEARCH

```

```

elif state == State.FOUND_SODER:
    if char == 'ж': state = State.FOUND_SODERZH
    else: state = State.SEARCH
elif state == State.FOUND_SODERZH:
    if char == 'а': state = State.FOUND_SODERZHA
    else: state = State.SEARCH
elif state == State.FOUND_SODERZHA:
    if char == 'н': state = State.FOUND_SODERZHAN
    else: state = State.SEARCH
elif state == State.FOUND_SODERZHAN:
    if char == 'и': state = State.FOUND_SODERZHANI
    else: state = State.SEARCH
elif state == State.FOUND_SODERZHANI:
    if char == 'е': state = State.FOUND_SODERZHANIE
    else: state = State.SEARCH
elif state == State.FOUND_SODERZHANIE:
    if char == ' ' or not char.isalpha():
        results.append(("СОДЕРЖАНИЕ", page_num + 1, line_num, line.strip()))
        state = State.SEARCH
    else:
        state = State.SEARCH

return results

# Пример использования
pdf_path = "report.pdf" # Укажите путь к вашему PDF-файлу
mentions = find_toc_mentions(pdf_path)

if mentions:
    print("Найдены следующие упоминания:")
    for mention in mentions:
        word, page, line_num, line_text = mention
        print(f"Слово: {word}, Страница: {page}, Строка: {line_num}, Текст: {line_text}")
else:
    print("Упоминания 'оглавление' или 'содержание' не найдены.")

```

Листинг 3 – cogito-2.1.

```

import PyPDF2

def detect_table_of_contents(pdf_path):
    """
    Определяет, используется ли в PDF файле оглавление вместо содержания.
    Использует конечный автомат для определения.

    Args:
        pdf_path (str): Путь к PDF файлу.

    Returns:
        list: Список строк с номерами страниц, если использовано оглавление.
              Возвращает пустой список, если использовано содержание или если файл не найден.
    """
    try:
        with open(pdf_path, 'rb') as pdf_file:
            reader = PyPDF2.PdfReader(pdf_file)
            num_pages = len(reader.pages)

            # Проверяем наличие заголовка. Оглавление обычно начинается с заголовка

```

```

first_page = reader.pages[0]
text = first_page.extract_text()

# Простейшая реализация конечного автомата
# Ищем строку "Оглавление" или "Содержание" (с учетом регистра)
if "Оглавление" in text.upper() or "Содержание" in text.upper():
    # Находим оглавление. Пример: "Оглавление"
    title_line = next((line for line in text.splitlines() if "Оглавление" in line.upper()), None)

    if title_line:
        # Оцениваем, что это именно оглавление, на основе структуры текста
        # Это очень упрощенный пример, и его можно улучшить
        if "Страницы" in title_line: # Предполагаем, что в строке оглавления есть "Страницы"
            print("Обнаружено оглавление.")
            toc_lines = []
            for page_num in range(1, num_pages + 1):
                try:
                    page = reader.pages[page_num - 1]
                    page_text = page.extract_text()
                    # Ищем строку с номером страницы. Это тоже упрощенно.
                    page_number_line = next((line for line in page_text.splitlines() if str(page_num) in line), None)
                    if page_number_line:
                        toc_lines.append((page_number_line.strip(), page_num))
                except IndexError:
                    pass # Страница может быть не найдена.
            return toc_lines
        else:
            print("Похоже на оглавление, но отсутствует строка 'Страницы'.")
            return []
    else:
        print("Строка оглавления не найдена.")
        return []

# Если не оглавление, считаем, что есть содержание
print("Обнаружено содержание.")
return []

except FileNotFoundError:
    print(f"Файл не найден: {pdf_path}")
    return []

except Exception as e:
    print(f"Ошибка при обработке PDF: {e}")
    return []

# Пример использования
pdf_file_path = "report.pdf" # Замените на путь к вашему PDF файлу
toc = detect_table_of_contents(pdf_file_path)

if toc:
    print("Оглавление:")
    for line, page in toc:
        print(f"{line}, {page}")
else:
    print("Оглавление не найдено.")

```

Вывод

В данном разделе мной были получены результаты запросов.

4 Исследовательская часть

4.1 Тестирование

Таблица 4.1 — Результаты тестирования.

Модель	Вход	Выход
qwen3-coder	Отчёт по первой лабораторной работе	Случаи использования 'оглавление' вместо 'содержание' не найдены
devstral-2	Отчёт по первой лабораторной работе	В файле используется 'содержание' (не 'оглавление').
cogito-2.1	Отчёт по первой лабораторной работе	Упоминания 'оглавление' или 'содержание' не найдены.
gemma3	Отчёт по первой лабораторной работе	Обнаружено содержание. Оглавление не найдено.

4.2 Трудоёмкость

С помощью введённой модели вычислений (табл. 4.2) была определена трудоёмкость ранее полученных реализаций, результаты представлены в табл. 4.3.

Таблица 4.2 — Цены базовых операций.

Операции	Цена
+, -, =, ++, --, «, »	1
*, /, %, *=, /=	2
==, >=, <=, !=, >, <, &&,	1
[], {}	1

Таблица 4.3 — Трудоёмкость алгоритмов.

БЯМ	Трудоёмкость ЛС	Трудоёмкость ХС
qwen3-coder	$90n + 342$	$O(60n)$
devstral-2	$11n + 45$	$O(n)$
cogito-2.1	$37n + 68$	$O(23n)$
gemma3	$8n$	$O(10n)$

Вывод

В данном разделе было проведено тестирование программ, написанных БЯМ, и рассчитана их трудоёмкость.

ЗАКЛЮЧЕНИЕ

В ходе работы мной были написаны запросы для БЯМ для генерации решения задания по варианту, проведено тестирование этих решений и рассчитана их трудоёмкость. Полученные реализации возвращают адекватный результат, соответствующий заданию.

Таким образом, все задачи были выполнены, а поставленная цель достигнута.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Документация Оллэма. [Электронный ресурс] Режим доступа: <https://docs.ollama.com/> (дата обращения: 22.01.2026)
2. J. Friedl. Mastering Regular Expressions. / Jeffrey E.F. Friedl – O'Reilly, 1997. – 342с.